Project 2: Chandy-Lamport Snapshot Algorithm

Overview

For this project, you will implement the Chandy-Lamport algorithm for computing the global state of a distributed system. The global-state recording algorithm is superimposed on the underlying computation, running concurrently but without altering such computation.

Chandy-Lamport Snapshot Algorithm Summary

Starting the snapshot process on a server:

- Record its local state.
- Send marker messages on all outbound interfaces.

When you receive a marker message:

- If you haven't started the snapshot process yet, record your local state and send marker messages on all other interfaces.
- Start recording messages you receive on all other interfaces.
- Stop recording messages you receive on this interface.

Terminate when all servers have received marker messages on all interfaces.

The number of tokens must be preserved in the snapshots.

The algorithm makes the following assumptions:

- There are no failures and all messages arrive intact and only once.
- The communication channels are unidirectional and FIFO ordered.
- There is a communication path between any two processes in the system.
- Any process may initiate the snapshot algorithm.
- The snapshot algorithm does not interfere with the normal execution of the processes.
- Each process in the system records its local state and the state of its incoming channel.

Your global snapshot algorithm will be implemented on top of a token-passing system, similar to the ones presented in the paper.

Key Idea

Servers send **marker messages** to each other.

- Markers mark the beginning of the snapshot process on the server, and
- Act as a barrier (stopper) for recording messages.

Project Setup

The starting code for this project can be downloaded here.

Run the following command to extract and set up the project environment:

```
tar -zxvf chandy_lamport.tgz
```

A brief description of the code base is as follows:

File	Description
`server.go`	A process in the distributed algorithm
`simulator.go`	A discrete-time simulator
`logger.go`	A logger that records events executed by the system (useful for debugging)
`common.go`	Debug flag and common message types used in the server, logger, and simulator
`snapshot_test.go`	Tests you need to pass
`syncmap.go`	Implementation of thread-safe map
`queue.go`	Simple queue interface implemented using lists in Go
`test_common.go`	Helper functions for testing
`test_data/`	Test case inputs and expected results

Of these files, you only need to make changes to `server.go` and `simulator.go`.

Your task is to implement the functions that say `TODO: IMPLEMENT ME`, adding fields to the surrounding classes if necessary.

However, some other files also contain information that will be useful for your implementation or debugging, such as the debug flag in `common.go` and the thread-safe map in `syncmap.go`.

The implementation of this project uses a discrete-time simulator to order events. The simulator manages servers and injects events into the system, while servers implement the snapshot algorithm.

States that the simulator needs to keep track of:

- Time
- Topology
- Channels to signal the completion of the snapshots

States that servers need to keep track of:

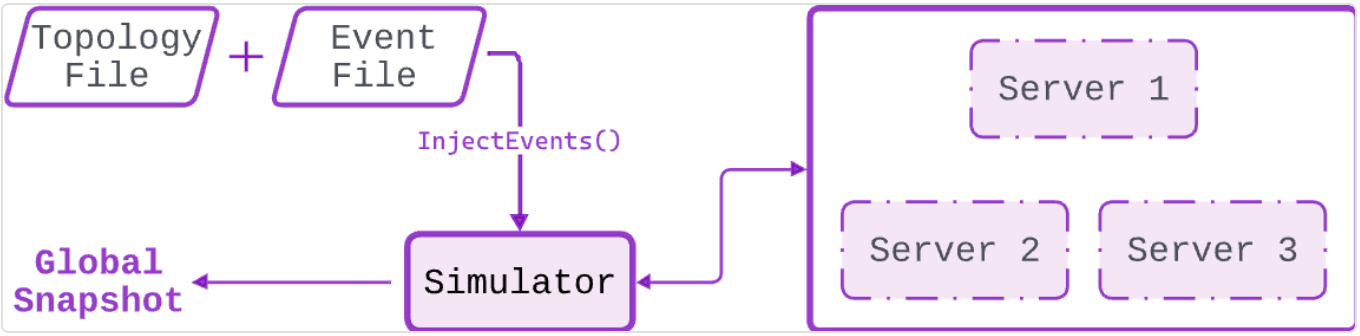
- Local state
- Neighbors
- Which interfaces received markers
- Recorded messages

Useful Tips

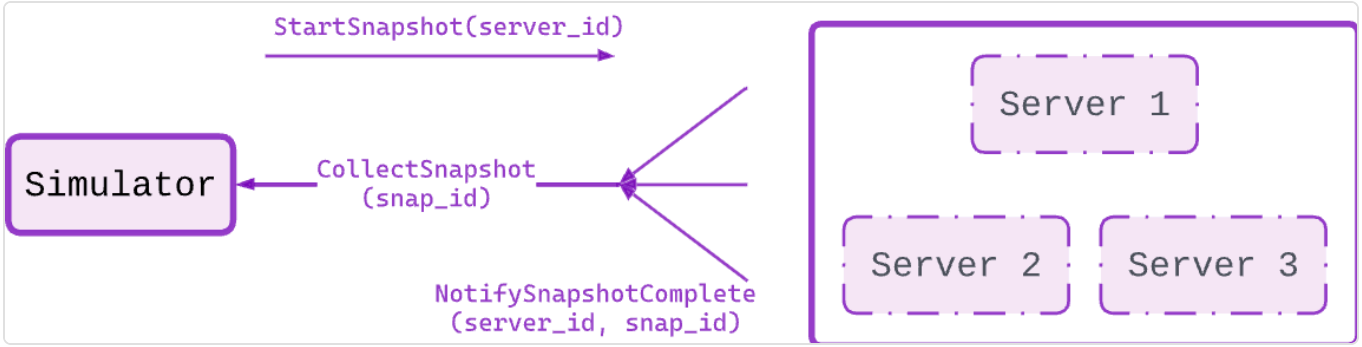
Below is an example of how to start exploring the project. However, feel free to navigate the codebase in any sequence that works for you.

1. Start by understanding how the simulator builds a topology and simulates events for a given test case.

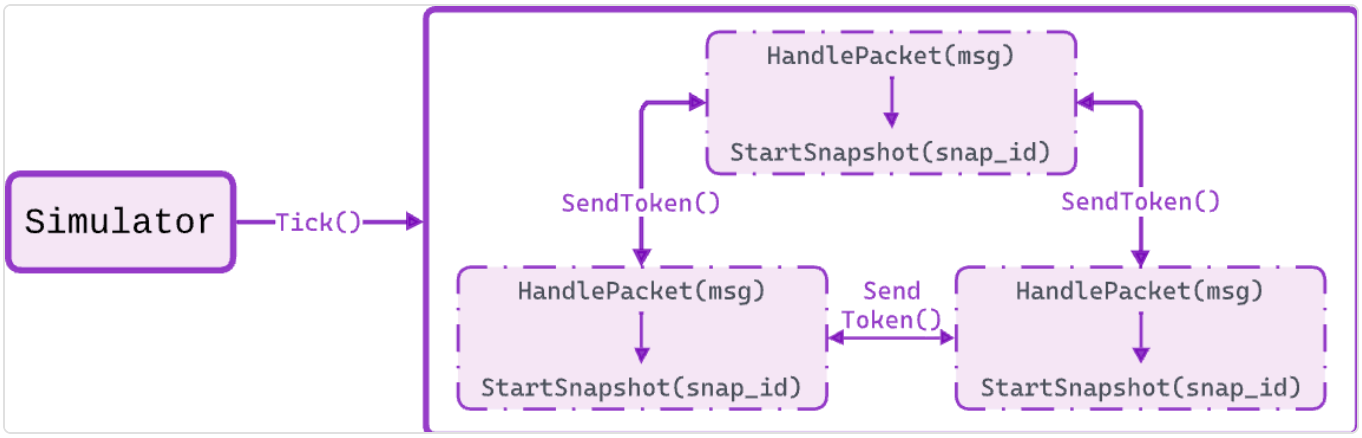
- `test_common.go` contains functions that parse topologies (`test_data/*.top`) and inject events (`test_data/*.events`) into the simulator.
- `snapshot_test.go` specifies how test cases are being run.



1. Next, try to understand the simulator's implementation (see `simulator.go`), the role it plays in this project, and the methods it uses to interact with server modules.



1. Finally, try to understand the server's implementation (see `server.go`), the methods it uses to take a local snapshot, as well as functions to communicate with each other.



Start with the base case, i.e. the simplest case:

- One global snapshot at a time
- Test regularly; you should pass the test case for 2 nodes first, then 3 nodes, and so on.
- Add one more complexity at a time.

Key idea: start simple, then build up.

- Phase 1: Single snapshot at a time
- Phase 2: Concurrent global snapshots

Track execution using printing statements to understand the code flow, e.g., print out the server state before and after implementing the `HandlePacket()` and `startSnapshot()` functions after each tick of the stimulator.

Testing

Test cases can be found in `test_data/`.

To run individual test cases, do:

```
go test -race -run 2NodesSimple
```

Example output:

```
>> Running test '2nodes.top', '2nodes-simple.events'
>> PASS
>> ok _/path/to/chandy-lamport 0.006s
```

Replace `2NodesSimple` with the specific test case intended to run. For example, to run both the `Test3NodesMultipleMessages` and `Test3NodesBidirectionalMessages` test cases, do:

```
go test -race -run 3NodesMultiple
```

Example output:

```
>> Running test '3nodes.top', '3nodes-simple.events'
>> Running test '3nodes.top', '3nodes-bidirectional-messages.events'
>> PASS
>> ok _/path/to/chandy-lamport 0.009s
```

To run all test cases, do:

```
go test -race
```

Example output:

```
>> Running test '2nodes.top', '2nodes-simple.events'
>> Running test '2nodes.top', '2nodes-message.events'
>> Running test '3nodes.top', '3nodes-simple.events'
>> Running test '3nodes.top', '3nodes-bidirectional-messages.events'
>> Running test '8nodes.top', '8nodes-sequential-snapshots.events'
>> Running test '8nodes.top', '8nodes-concurrent-snapshots.events'
```

```
>> PASS
>> ok _/path/to/chandy-lamport 0.012s
```

Grading

The TAs will grade your implementation with the following:

Public Test Cases

There are 6 public test cases in `snapshot_test.go`:

- **`Test2NodesSimple`**:
  - topology → `2nodes.top`
  - events → `2nodes-simple.events`
  - expected result → `2nodes-simple.snap`
- **`Test2NodesSingleMessage`**:
  - topology → `2nodes.top`
  - events → `2nodes-message.events`
  - expected result → `2nodes-message.snap`
- **`Test3NodesMultipleMessages`**:
  - topology → `3nodes.top`
  - events → `3nodes-simple.events`
  - expected result → `3nodes-simple.snap`
- **`Test3NodesMultipleBidirectionalMessages`**:
  - topology → `3nodes.top`
  - events → `3nodes-bidirectional-messages.events`
  - expected result → `3nodes-bidirectional-messages.snap`
- **`Test8NodesSequentialSnapshots`**:
  - topology → `8nodes.top`
  - events → `8nodes-sequential-snapshots.events`
  - expected results →
    - `8nodes-sequential-snapshots0.snap`
    - `8nodes-sequential-snapshots1.snap`
- **`Test8NodesConcurrentSnapshots`**:
  - topology → `8nodes.top`
  - events → `8nodes-concurrent-snapshots.events`
  - expected results →
    - `8nodes-concurrent-snapshots0.snap`
    - `8nodes-concurrent-snapshots1.snap`

- `8nodes-concurrent-snapshots2.snap`
- `8nodes-concurrent-snapshots3.snap`
- `8nodes-concurrent-snapshots4.snap`

Private Test Cases

There is 1 private test case not in `snapshot_test.go`:

- **`Test10NodesSimple`**:
  - topology → `10nodes.top`
  - events → `10nodes.events`
  - expected result → not provided

Submission Instructions

File contents: **ALL FILES** of the project.

1. Move to the parent directory of the project folder.

For example, if the project folder is located at `~/lab2_chandy_lamport`, then run:

```
cd ..
```

1. Create the submission tarball:

```
tar -zcvf lab2_chandy_lamport_<GROUP_NUMBER>.tgz lab2_chandy_lamport
```

1. Make sure to include all project files.

The TAs will run the following command to extract submissions:

```
tar -zxvf lab2_chandy_lamport_<GROUP_NUMBER>.tgz
```

Do **not** leave any code for printing/debugging. Additional text output in the test result may significantly distract grading, so it will deduct minor points from your grade.

Do not include extra files unrelated to the project.